# 综合练习

## 人员

杨俊彦、褚锦轩、陈欣妙、李锦澍、阮文璋、袁晨峻 到课

## 上周作业检查

上周作业链接: https://cppoj.kids123code.com/contest/3299

比赛概况 题目列表 选择题列表 提交记录 实时榜单 选择题排行榜 编辑比赛

王向东老师周日八点半C++综合练习

刷新

| # | 用户名 | 姓名 | 编程分 | 时间 | A | B | C |
|---|---|---|---|---|---|---|---|
| 1 | yuanchenjun | 袁晨峻 | 300 | 6054 | 100 | 100 | 100 |
| 2 | chenxinmiao | 陈欣妙 | 300 | 7490 | 100 | 100 | 100 |
| 3 | lijinshu | 李锦澍 | 300 | 7578 | 100 | 100 | 100 |
| 4 | yangjunyan | 杨俊彦 | 300 | 11567 | 100 | 100 | 100 |
| 5 | chujinxuan | 褚锦轩 | 199 | 2348 | 100 | 27 | 72 |
| 6 | suitianyi | 隋天乙 | 100 | 890 | 100 | | |
| 7 | liuyichen | 刘奕辰 | 100 | 8208 | | | 100 |

## 本周作业

https://cppoj.kids123code.com/contest/3408 (课上讲了 A ~ B 题, 课后作业是 B 题必做)

## 课堂表现

今天的 B 题稍微会绕一些, 其实知识点很简单, 就是前缀和 + map维护, 同学们课下再好好复盘一下这道题。

## 课堂内容

**Find Permutation (上周作业)**

根据题目输入的大小关系建图, 跑拓扑排序, 看是否存在唯一的拓扑序即可

如何看是否有唯一的拓扑序: 任意时刻队列中的数不能大于等于 2

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 2e5 + 5;
vector<int> vec[maxn];
int deg[maxn];
int f[maxn];

int main()
```

```
{
  int n, m; cin >> n >> m;
  set<pair<int,int>>s;
  while (m -- ) {
    int a, b; cin >> a >> b; s.insert({a,b});
  }
  for (pair<int,int> it : s) {
    int a = it.first, b = it.second;
    vec[a].push_back(b); ++deg[b];
  }

  queue<int> q;
  for (int i = 1; i <= n; ++i) {
    if (!deg[i]) q.push(i);
  }
  int cnt = 0;
  while (!q.empty()) {
    if ((int)q.size() >= 2) { cout << "No" << endl; return 0; }
    int u = q.front(); q.pop();
    ++cnt; f[u] = cnt;
    for (int i : vec[u]) {
      --deg[i];
      if (!deg[i]) q.push(i);
    }
  }

  cout << "Yes" << endl;
  for (int i = 1; i <= n; ++i) cout << f[i] << " ";
  cout << endl;
  return 0;
}
```

**Distance Sequence**

f[i][j]: 考虑前 i 项, 第 i 项结尾为 j 时, 一共有多少符合题意的方案

答案: f[n][1] + f[n][2] + ... + f[n][m]

转移: f[i][j] = (f[i-1][1] + f[i-1][2] + ... + f[i-1][j-K]) + (f[i-1][j+K] + f[i-1][j+K+1] + ... + f[i-1][m])

```
  状态是 O(n^2) 的，转移是 O(n) 的，所以会超时

  因此，转移过程可以用前缀和来优化
```

```
#include <bits/stdc++.h>
#define int long long

using namespace std;
```

```
const int N = 1000 + 5, M = 5000 + 5;
const int mod = 998244353;
int f[N][M], p[N][M];

int get_sum(int id, int l, int r) {
  if (l > r) return 0;
  return (p[id][r]-p[id][l-1]+mod)%mod;
}

signed main()
{
  int n, m, K; cin >> n >> m >> K;
  for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
      if (i == 1) f[i][j] = 1;
      else if (K) f[i][j] = (get_sum(i-1,1,j-K) + get_sum(i-1,j+K,m)) % mod;
      else f[i][j] = get_sum(i-1,1,m);
    }
    for (int j = 1; j <= m; ++j) p[i][j] = (p[i][j-1]+f[i][j]) % mod;
  }

  int res = 0;
  for (int i = 1; i <= m; ++i) res = (res + f[n][i]) % mod;
  cout << res << endl;
  return 0;
}
```

**Rem of Sum is Num**

对于任意符合要求的 [l,r] 连续子序列, 应该满足 (p[r] - p[l-1]) % mod == r - l + 1, (r-l+1 < mod)

等式变化一下, 即: (p[r] - r) % mod == (p[l-1] - (l-1)) % mod, (r-l+1 < mod)

设定 c[i] = p[i] - i

枚举右边界 r, 此时就是看 c[r-mod+1] ~ c[r-1] 中, 有多少项等于 c[r], 这个可以在循环过程中用 map 进行维护

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int maxn = 2e5 + 5;
int w[maxn], p[maxn], c[maxn];

signed main()
{
  int n, K; cin >> n >> K;
  for (int i = 1; i <= n; ++i) {
    cin >> w[i], p[i] = p[i-1] + w[i], c[i] = (p[i] - i + K) % K;
  }
```

```
    int res = 0;
    map<int,int> mp; mp[p[0]-0] = 1;
    for (int i = 1; i <= n; ++i) {
      if (i-K >= 0) mp[c[i-K]]--;
      res += mp[c[i]];
      mp[c[i]]++;
    }
    cout << res << endl;
    return 0;
  }
```